Vignesh Damodharan
EDGE IT ANALYST
DTCC - CHENNAI

Insurance Management System Documentation

Project Title:

Insurance Management System

Project Done By : Vignesh Damodharan – EDGE IT ANALYST – DTCC

Project Title Overview:

The **Insurance Management System** is a digital platform that enables customers to apply for, manage, and track various types of insurance policies. The system also allows users to submit claims and make payments securely. Admins have control over the management of policies, claims approval, and overall user administration. The platform is built using **Java Spring Boot** for the backend, **Angular** for the frontend, and integrates **Docker**, **Jenkins**, and **AWS EC2** for seamless deployment and CI/CD automation.

Project Git Hub URL : https://github.com/Vignesh-D-2003/Insurance-Management.git

Key Capabilities:

- **Policy Management**:
  - Customers can view, apply for, and manage various types of insurance policies including bike, car, health, and term life insurance. Admins can create, update, or delete policies.
- **Claims Submission**:
  - Customers can submit claims for their active insurance policies by providing necessary details and uploading relevant files. Admins can view, approve, or reject claims submitted by customers.
- **User Roles & Authentication**:
  - **Customer Role**: Allows customers to apply for insurance policies, track claims, and view payment status.
  - **Admin Role**: Allows admins to manage policies, view customer claims, and approve or reject claims.
  - Authentication is handled via **Spring Security** with role-based access control (RBAC).
- **Policy Document Generation**:
  - Upon successful payment, a **policy document** is automatically generated with a unique policy number.

- **User Dashboard**:

  - **Customer Dashboard**: Displays active policies, claims, and payment details.
  - **Admin Dashboard**: Provides views for managing policies and claims submitted by customers.

Skill Sets Covered:

1. Angular: Policy dashboards, claim tracking modules, document upload integration.
2. Spring Boot: APIs for policy management, claims workflow, and JWT authentication.
3. DevOps: Ansible deployment automation, Docker container build, AWS EC2 configuration.

Expected Outcomes:

1. Learn domain-specific application workflows.
2. Implement secure document and data management.

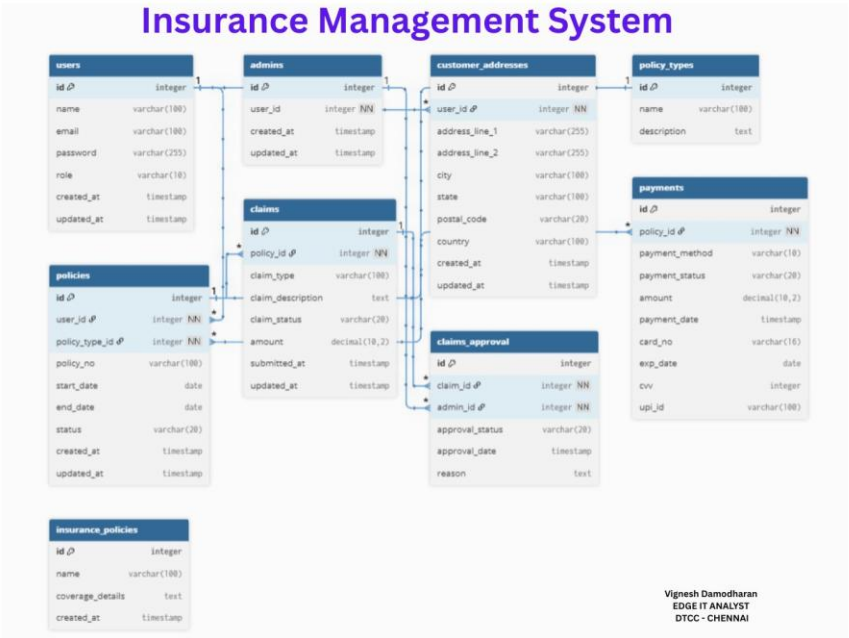
Technology Stack

- **Backend**: Java, Spring Boot, Spring Security
- **Frontend**: Angular, HTML, CSS, JavaScript
- **Database**: H2 (In Memory Database)(for storing policies, claims, and user data)
- **CI/CD**: Jenkins, Docker, AWS EC2, Ansible
- **Monitoring**: Prometheus, Grafana
- **Authentication**: JWT-based Authentication with Role-Based Access Control (RBAC)

Deployment Workflow

1. Create the Database and Write SQL Schema

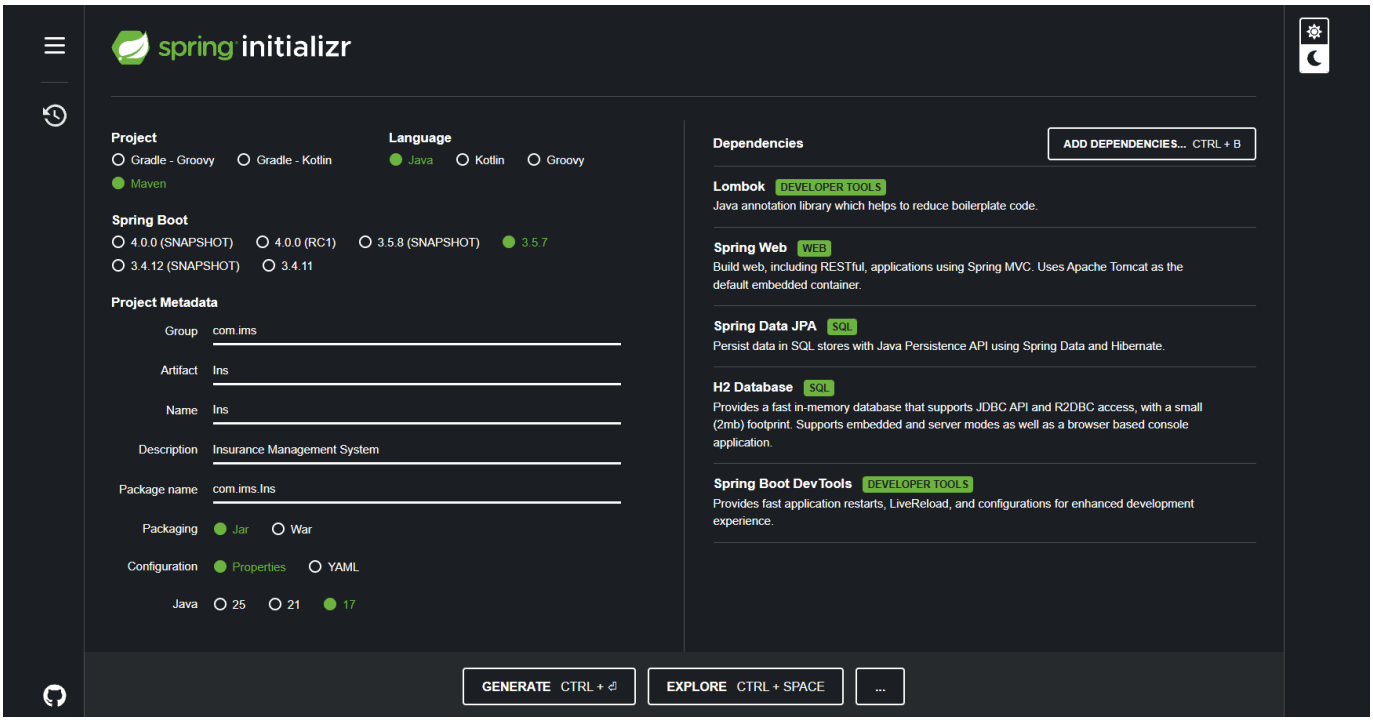
1. **Create Database Schema**:
   - Create a DBML Diagram Using dbdiagram.io
   - Write the SQL code for creating the database tables based on the DBML schema provided earlier.



2. Backend Development Setup

Step 1: Create New Maven Spring Boot Project

1. **Spring Initializer**:
   - Open the Browser search Spring Initializr (https://start.spring.io/)and create a new project.
     - **Project**: Maven
     - **Language**: Java
     - **Spring Boot Version**: 3.5.7
     - **Dependencies**:
       - Spring Web
       - Spring Data JPA
       - Spring Security
       - H2 Database (or your preferred DB)
       - Spring Boot DevTools
       - Swagger UI
       - Lombok (optional for reducing boilerplate)

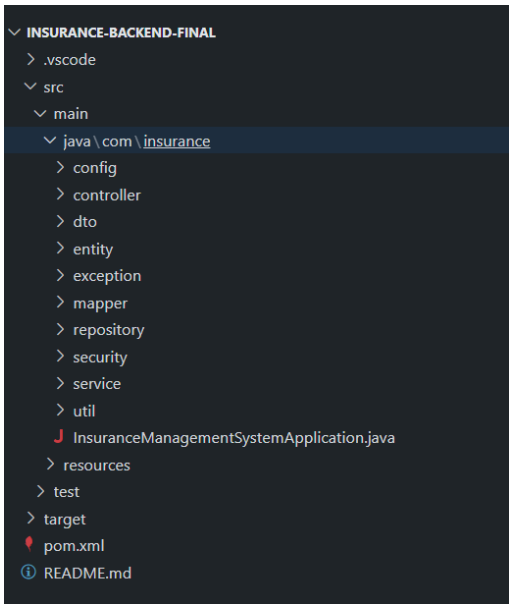


Spring Initializr Webiste

2. **Download and Extract the Project**:
   - Click **Generate**, download the zip file, and extract it.

Step 2: Project Structure Setup

1. **Create Java Packages**:
   - Inside the `src/main/java/com/ims/Ins`, create the following packages:
     - `dao` (Data Access Objects)
     - `dto` (Data Transfer Objects)
     - `controller` (REST Controllers)
     - `service` (Business Logic Layer)
     - `entity` (Database Entities)
     - `mapper` (Object-Relational Mapping, e.g., DTO to Entity mapping)



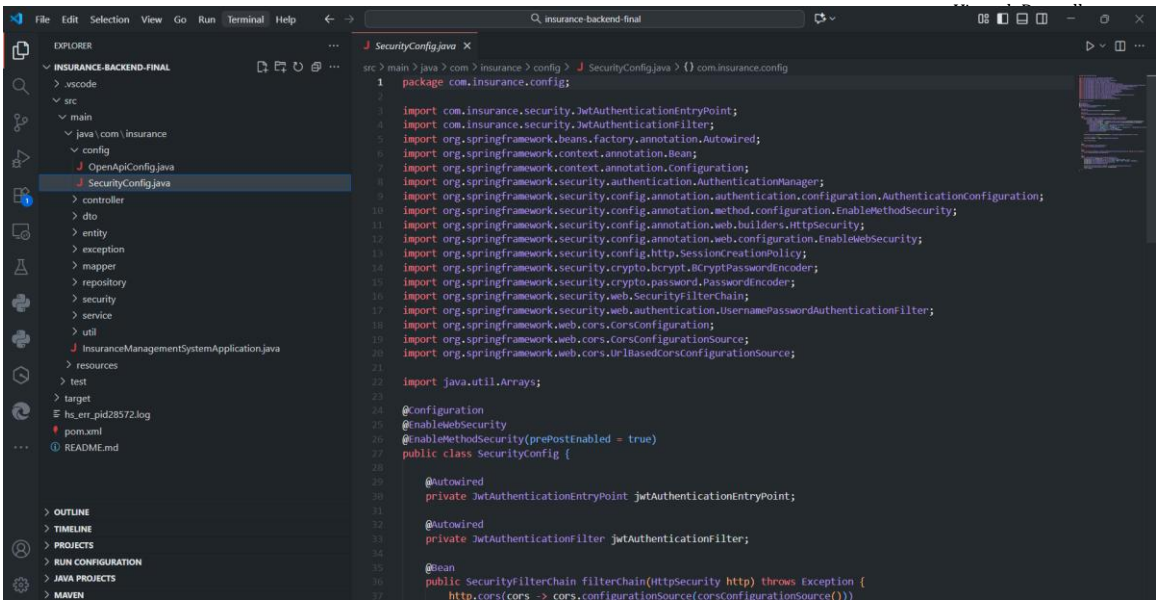
Backend File Structure

Step 3: Spring Security with Role-Based Authentication

1. **Add Security Configuration**:
   - Configure **Spring Security** for role-based access control.
     - Define two roles: `ADMIN` and `CUSTOMER`.
     - Create an authentication mechanism (e.g., using JWT or Basic Authentication).

   Screenshot to Capture: Screenshot of `SecurityConfig.java` file with role-based authentication.

2. **Create Admin and Customer Roles**:
   - Create database entries for **admin** and **customer** roles.



Screenshot of `SecurityConfig.java` file with role-based authentication.

Step 4: Create REST API Endpoints

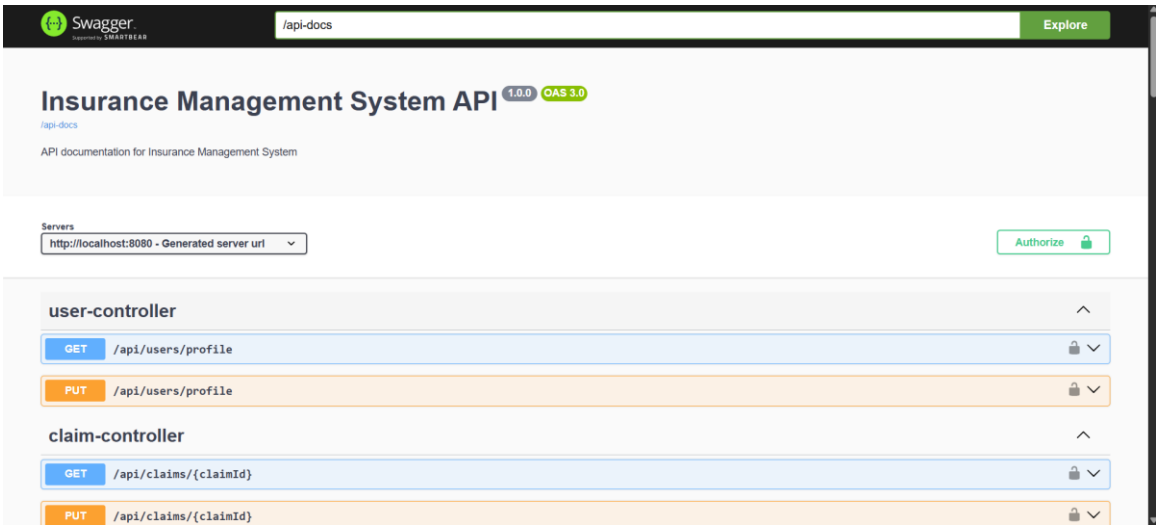
1. **Create Controllers**:
   - Implement REST APIs for managing policies, claims, payments, and admin functionalities.

   Example API:

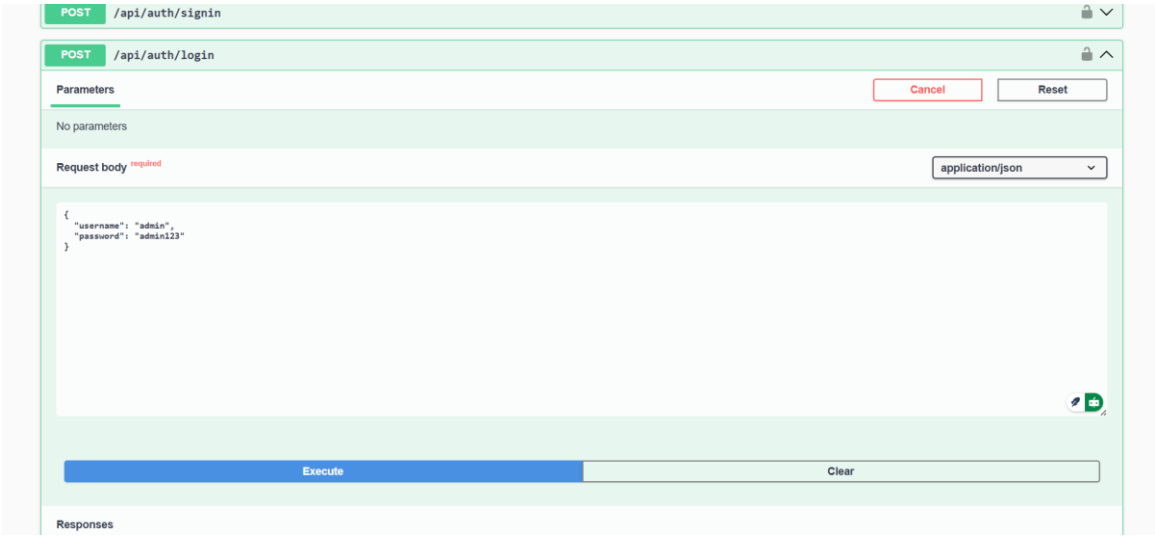
   - `GET /policies` (for customers to view policies)
   - `POST /claims` (for submitting claims)
   - `POST /policies/{id}/approve` (admin approves a policy)
2. **Swagger Setup**:
   - Add Swagger UI to the project for API documentation.

   Terminal Command to Run Spring Boot Application:

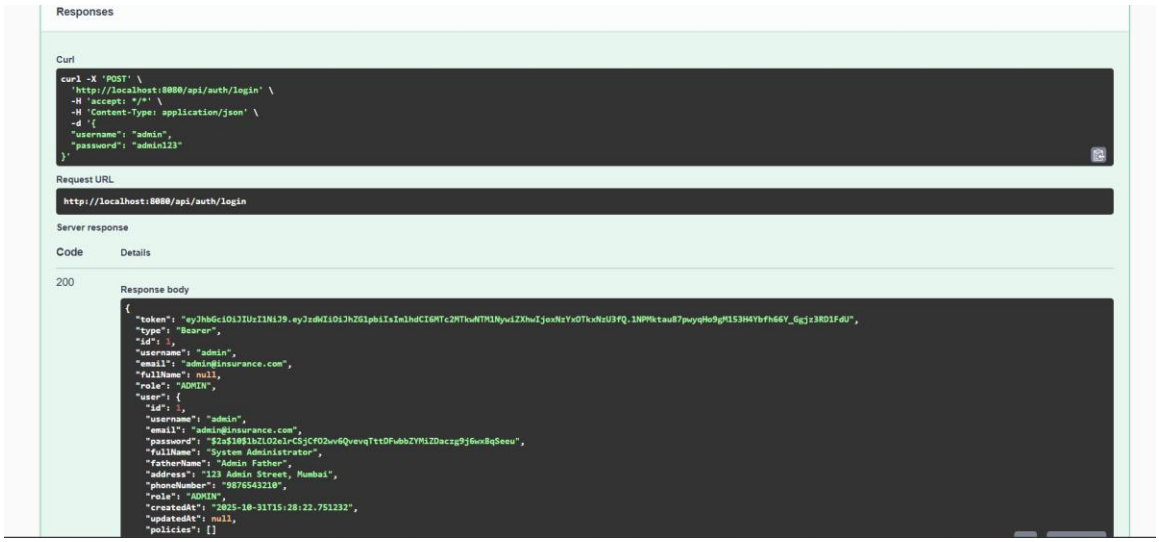
```
mvn spring-boot:run
```



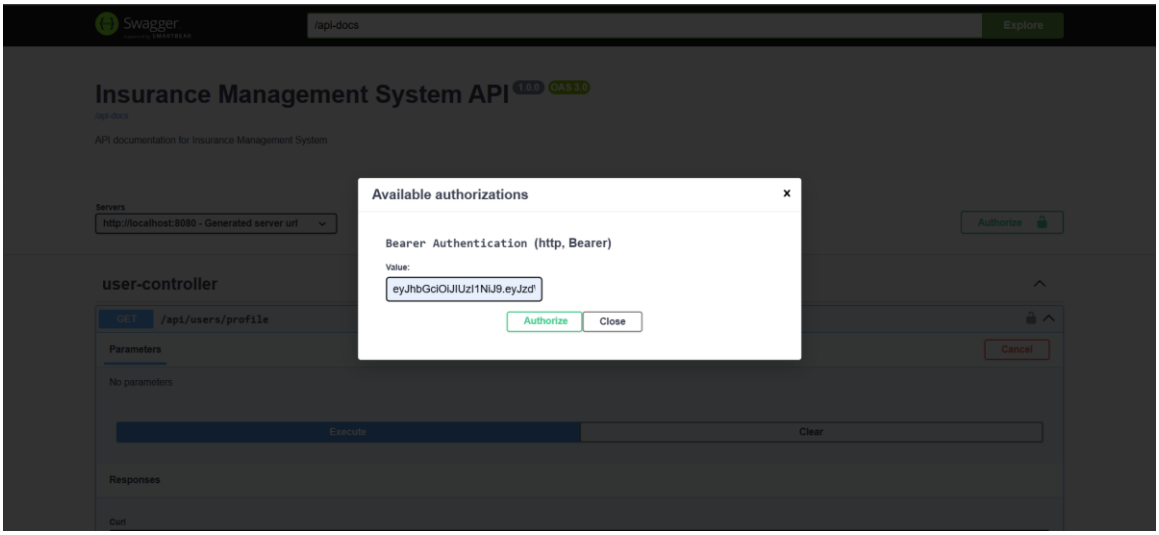
Swagger UI With Backend Running with End Points



POST /api/auth/login



Response POST Admin Login



Authorize with JWT Token to Check All end Point

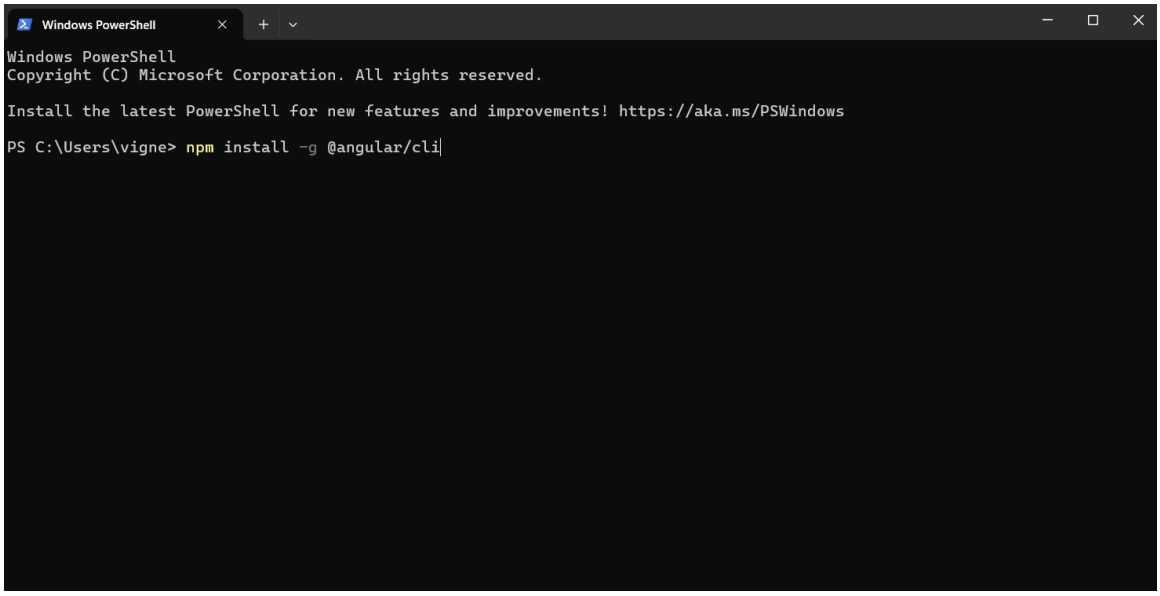
3. Angular Setup

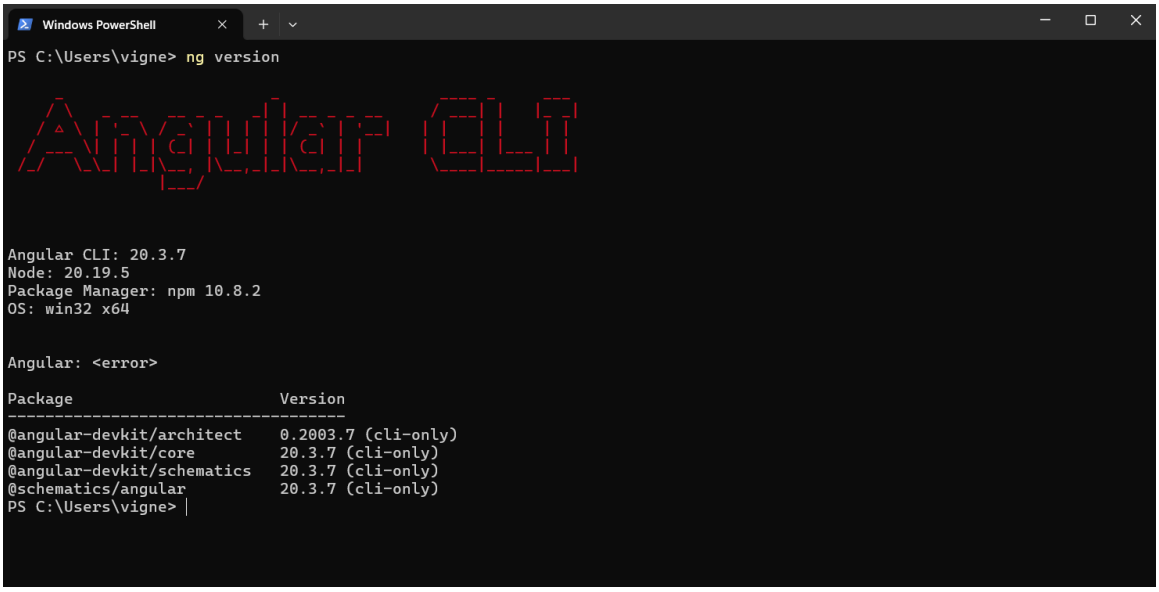
Step 1: Install Angular

1. **Install Angular CLI**:
   - Install Angular globally on your machine.

   Terminal Command:

   ```
   npm install -g @angular/cli
   ```



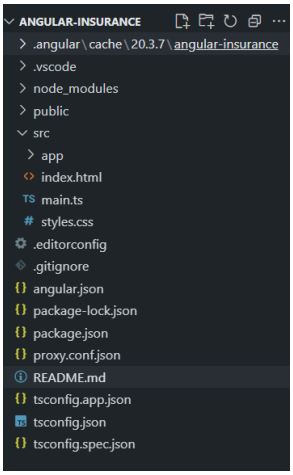


Step 2: Create Angular Project

1. **Generate Angular Project**:
   - Create a new Angular project using the Angular CLI.

Terminal Command:

```
ng new insurance-management-frontend
```



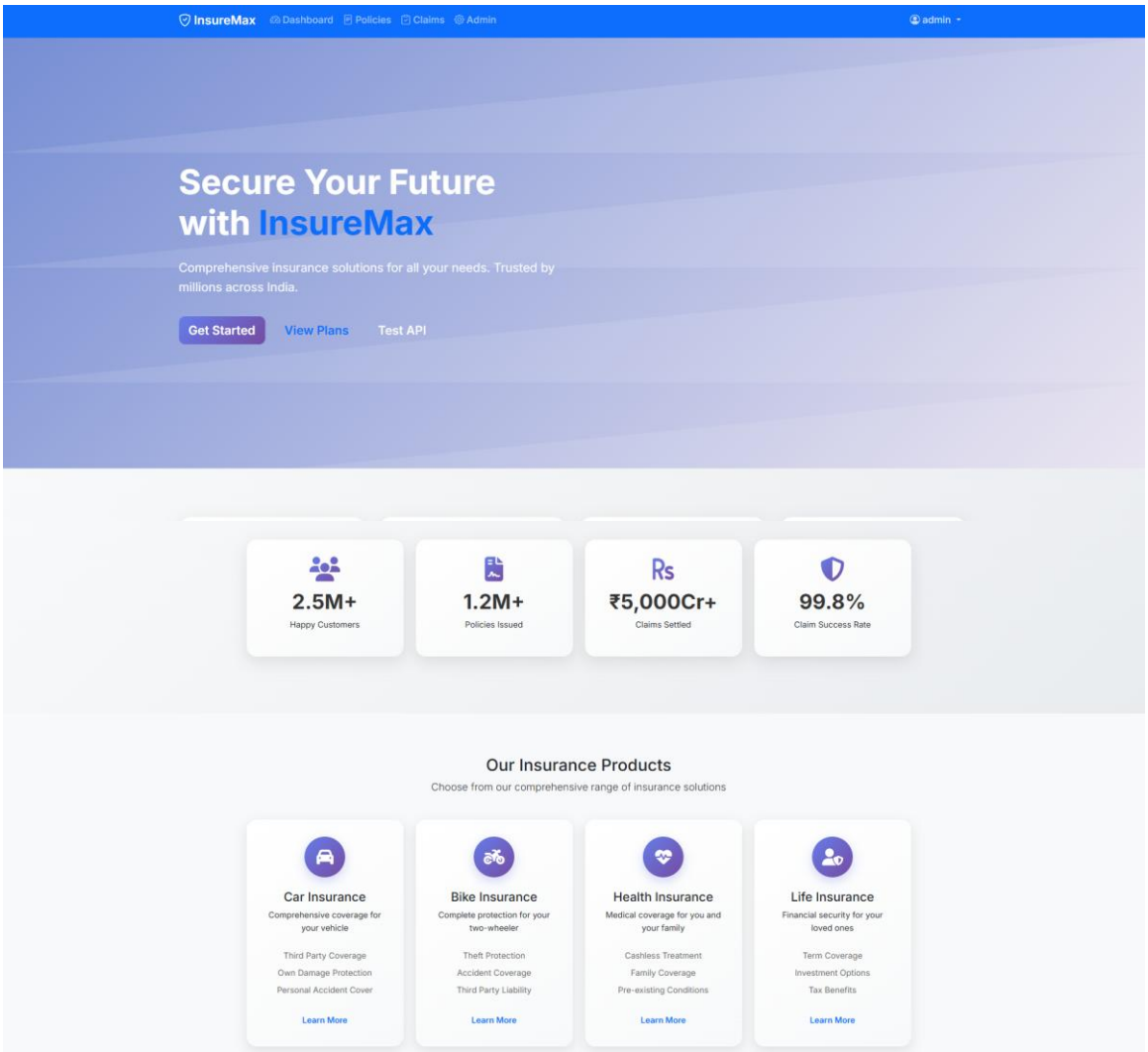
Frontend File Structure

Step 3: Create Components for Customer and Admin

1. **Generate Components**:
   - Generate components for customer dashboards, policy views, claim forms, etc.

Terminal Command:

```
ng generate component customer-dashboard
ng generate component admin-dashboard
```



Front End UI

4. Dockerizing the Application

Step 1: Dockerize Backend (Spring Boot)

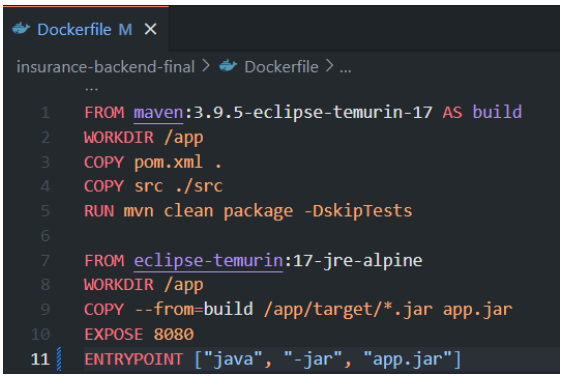
1. **Create Dockerfile**:
   - Create a `Dockerfile` for the Spring Boot application to run as a container.

Example Dockerfile:

```
FROM openjdk:11-jre-slim
COPY target/insurance-management-*.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Terminal Command:

```
docker build -t insurance-management-backend .
```



Step 2: Dockerize Frontend (Angular)

1. **Create Dockerfile for Angular**:
   - Create a `Dockerfile` for Angular.

Example Dockerfile:

```
FROM node:14 AS build
WORKDIR /app
COPY . .
RUN npm install
RUN ng build --prod
FROM nginx:alpine
COPY --from=build /app/dist/insurance-management-frontend
/usr/share/nginx/html
```

Terminal Command:

```
docker build -t insurance-management-frontend .\
```

Dockerfile

insurance-frontend-angular > Dockerfile > ...

You, yesterday | 1 author (You)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist/angular-insurance/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Step 3: Create Docker Compose File

1. **Create docker-compose.yml**:
   - Use a Docker Compose file to define both backend and frontend containers.

Example docker-compose.yml:

```
version: '3'
services:
  backend:
    image: insurance-management-backend
    ports:
      - "8080:8080"
  frontend:
    image: insurance-management-frontend
    ports:
      - "80:80"
```

Terminal Command:

```
docker-compose up -build
```

docker-compose.yml

```yaml
version: '3.8'

services:
  backend:
    build:
      context: ./insurance-backend-final
      dockerfile: Dockerfile
    container_name: insurance-backend
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - SPRING_DATASOURCE_URL=jdbc:h2:mem:insurance_db
      - SPRING_DATASOURCE_USERNAME=sa
      - SPRING_DATASOURCE_PASSWORD=password
    networks:
      - insurance-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/api/statistics/dashboard"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s

  frontend:
    build:
      context: ./insurance-frontend-angular
      dockerfile: Dockerfile
    container_name: insurance-frontend
    ports:
      - "4200:80"
    depends_on:
      - backend
```

5. CI/CD Setup with Jenkins

Step 1: Write Jenkins Pipeline File

1. **Create Jenkinsfile**:
   - Create a `Jenkinsfile` in the root of your repository.

Example Jenkinsfile:

```
pipeline {
 agent any
 stages {
  stage('Build') {
   steps {
    script {
     sh 'mvn clean install'
    }
   }
```

Jenkinsfile M ×

Jenkinsfile

You, 1 second ago | 1 author (You)

```
pipeline {
    agent any

    environment {
        // Docker Hub credentials (stored in Jenkins)
        DOCKER_HUB_CREDENTIALS = credentials('dockerhub-cred')
        DOCKER_HUB_USER = "${DOCKER_HUB_CREDENTIALS_USR}"
        DOCKER_HUB_PASS = "${DOCKER_HUB_CREDENTIALS_PSW}"

        // Docker image names
        FRONTEND_IMAGE = "${DOCKER_HUB_USER}/insurance-frontend:latest"
        BACKEND_IMAGE = "${DOCKER_HUB_USER}/insurance-backend:latest"

        // AWS EC2 instance details
        EC2_HOST = "ubuntu@54.164.59.117"   // ✅ Replace with your EC2 public IP
    }

    stages {

        stage('Checkout Code') {
            steps {
                echo "🔹 Checking out source code from GitHub..."
                git branch: 'main', url: 'https://github.com/Vignesh-D-2003/Insurance-Management.git'
            }
        }

        stage('Build Backend (Spring Boot)') {
            steps {
                echo "⚙️ Building Spring Boot backend..."
                dir('insurance-backend-final') {
                    sh 'mvn clean package -DskipTests'
                }
            }
        }

        stage('Build Frontend (Angular)') {
```

You, 1 second ago • Uncommitted changes

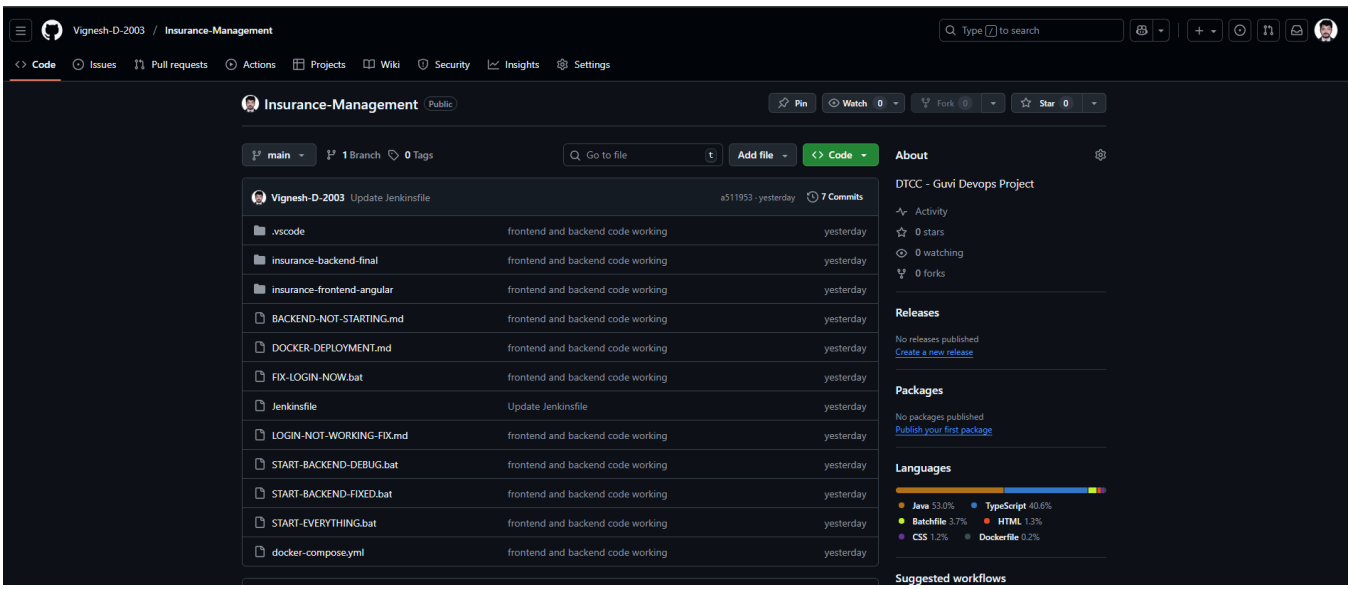
Screenshot Jenkins File

Step 2: Push Code to GitHub

1. **Create GitHub Repository**:
   - Create a new repository on GitHub.
2. **Push Code**:
   - Push your local project to GitHub.

Terminal Commands:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin < https://github.com/Vignesh-D-
2003/Insurance-Management.git>
git push -u origin master
```



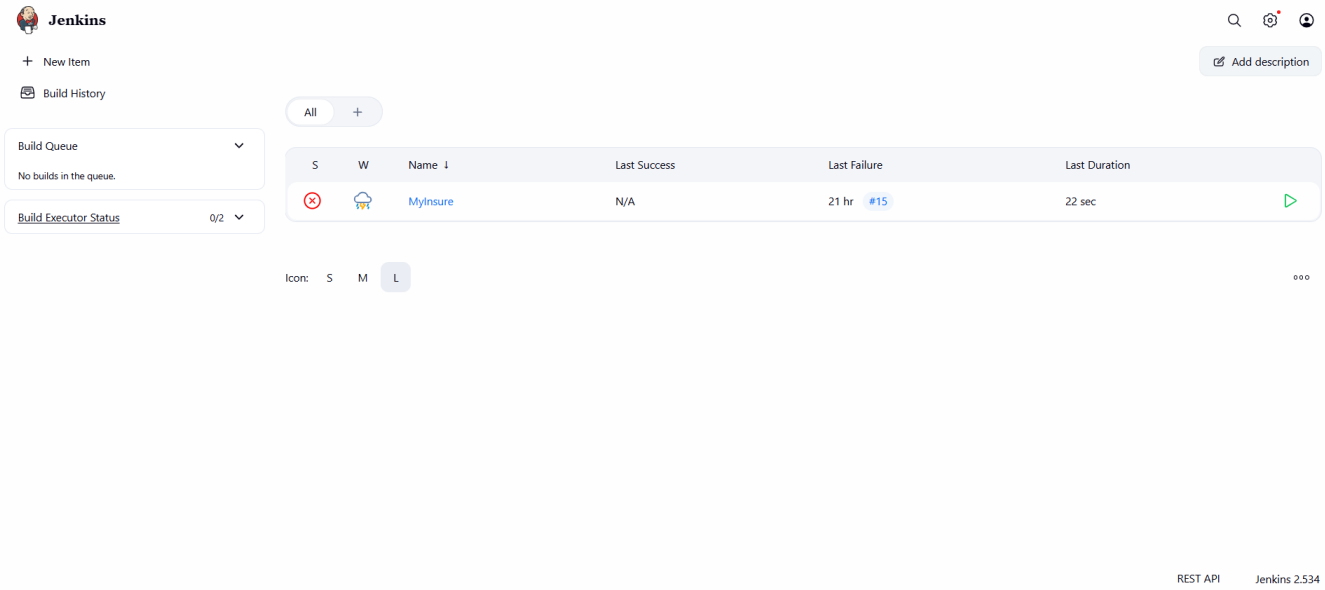
GitHub Project Repository

6. Set Up Jenkins CI/CD Pipeline

1. **Install Jenkins**:
   - Install Jenkins on your local machine or server.

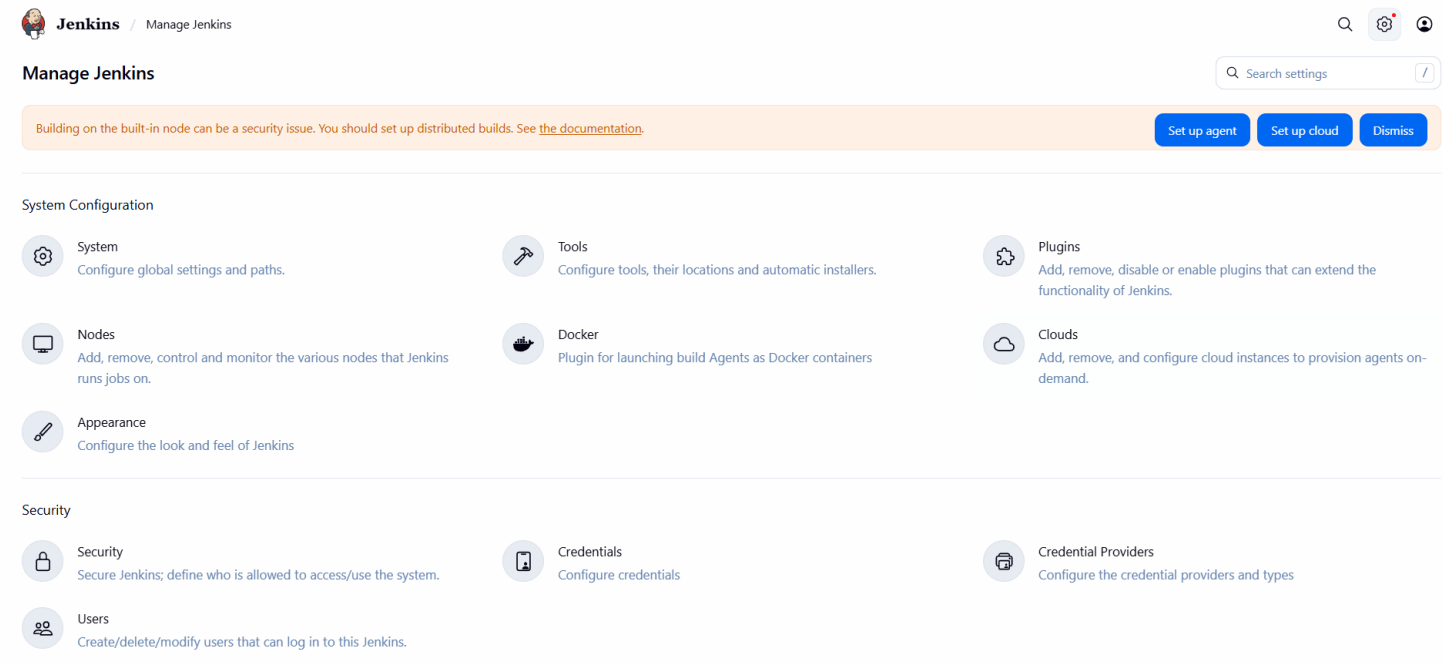
   Screenshot to Capture: Screenshot of Jenkins installation page.

2. **Configure Jenkins Pipeline**:
   - Set up Jenkins to pull code from your GitHub repository and run the pipeline.

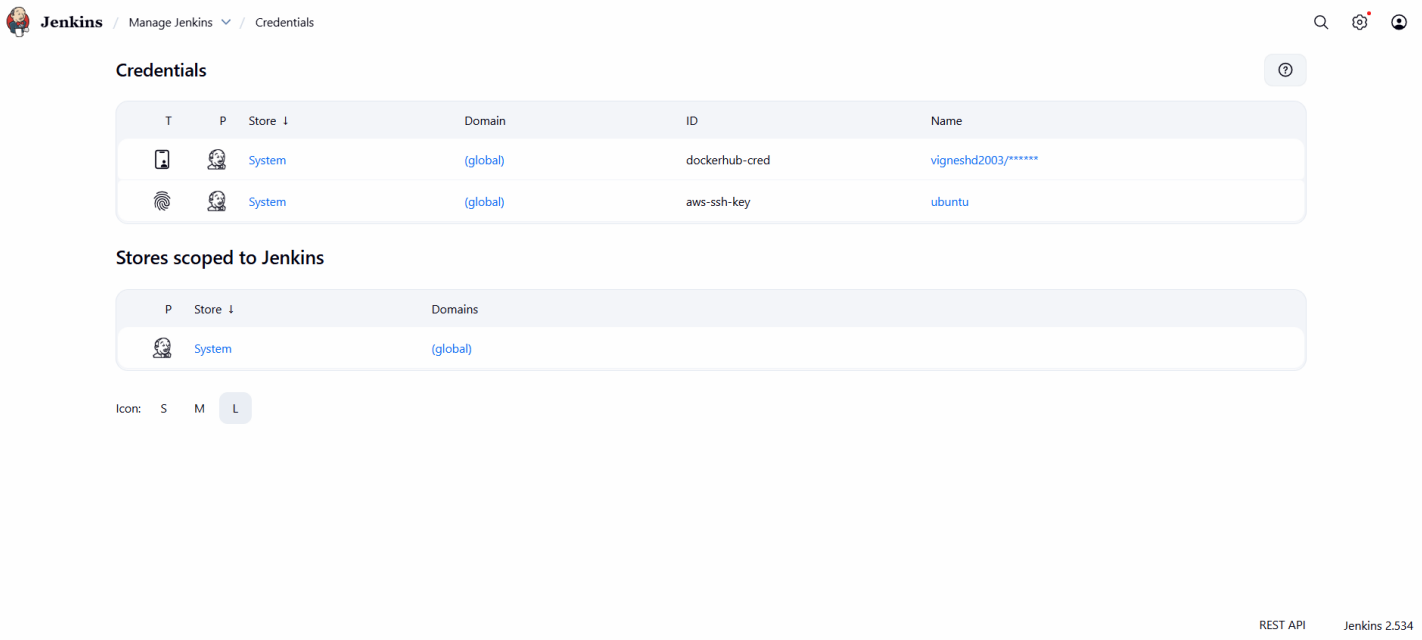


Vignesh Damodharan
EDGE IT ANALYST
DTCC - CHENNAI

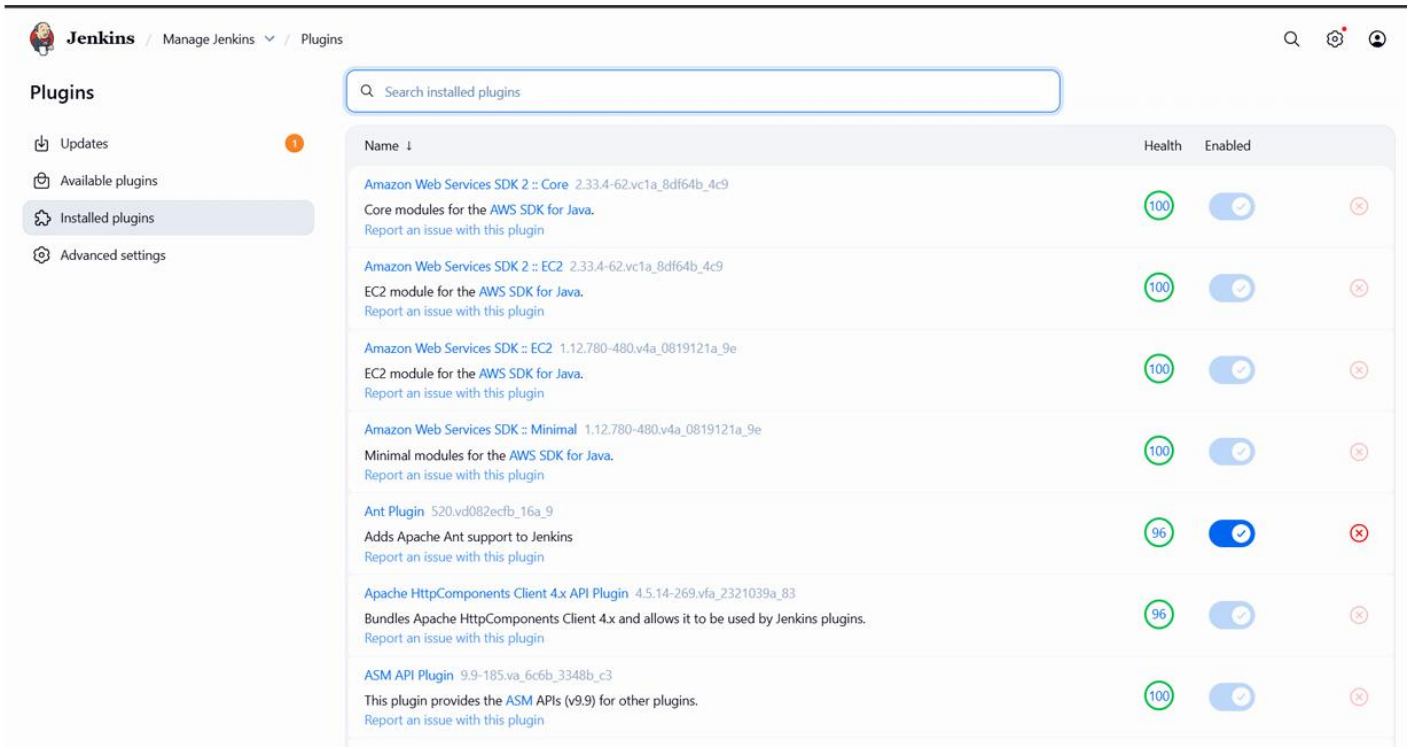
Jenkins Home Page



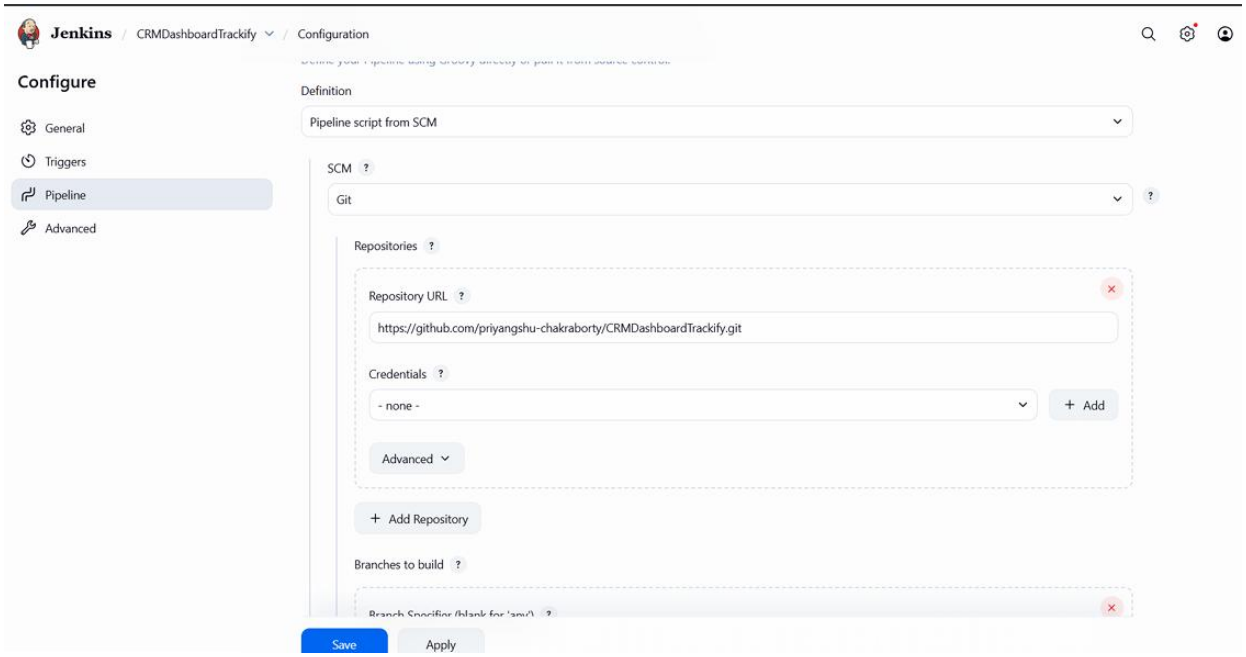
Go to Manage Jenkins



Create the Credentials used in your Jenkinsfile for docker credentials,aws credentials and ssh access credentials



Open Jenkins and download all the necessary plugins



Create a jenkins job with pipeline with configuration a. Select pipeline with SCM and Select Git for SCM b. Give the Github URL with the branch specified and Give the File name of your script which is generally Jenkinsfile. Then Save the Pipeline

7. AWS EC2 Setup with Ansible

Step 1: Install Ansible in WSL

1. **Install Ansible**:
   - Install Ansible on your WSL Ubuntu machine.

   Terminal Command:

```
sudo apt update
sudo apt install ansible
```

```
root@ip-172-31-26-41:~# sudo apt install ansible
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ansible-core python3-argcomplete python3-dnspython python3-kerberos
  python3-libcloud python3-lockfile python3-ntlm-auth python3-passlib
  python3-requests-ntlm python3-resolvelib python3-selinux python3-simplejson
  python3-winrm python3-xmltodict
Suggested packages:
  cowsay sshpass python3-trio python3-aioquic python3-h2 python3-httpx
  python3-httpcore python-lockfile-doc
The following NEW packages will be installed:
  ansible ansible-core python3-argcomplete python3-dnspython python3-kerberos
  python3-libcloud python3-lockfile python3-ntlm-auth python3-passlib
  python3-requests-ntlm python3-resolvelib python3-selinux python3-simplejson
  python3-winrm python3-xmltodict
0 upgraded, 15 newly installed, 0 to remove and 27 not upgraded.
Need to get 19.5 MB of archives.
After this operation, 315 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Step 2: Execute Ansible Code to Create EC2 Instance

1. **Run Ansible Playbook**:
   - Write an Ansible playbook to create an EC2 instance on AWS.

   Example Command:

```
ansible-playbook -i hosts aws_ec2_playbook.yml
```

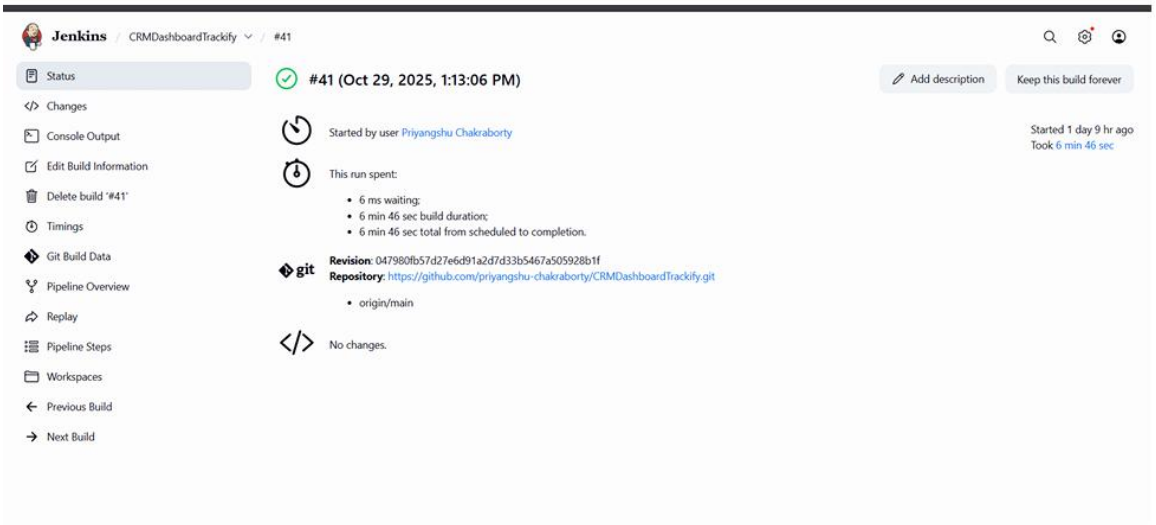
root@ip-172-31-26-41: /home/ansible

```yaml
---
- name: Provision EC2 instance on AWS
  hosts: localhost
  gather_facts: no
  tasks:

    - name: Launch a new EC2 instance
      ec2:
        key_name: "your-key-pair"  # Replace with your EC2 key pair name
        region: "us-east-1"  # Replace with the AWS region you want to use
        instance_type: "t2.micro"  # Choose the EC2 instance type
        image: "ami-0c55b159cbfafe1f0"  # Replace with the AMI ID of your choice (e.g., Amazon Linux 2 AMI)
        wait: yes
        group: "sg-0123456789abcdef0"  # Replace with your security group ID
        count: 1
        instance_tags:
          Name: "InsuranceAppEC2"  # Tag to identify your EC2 instance
        vpc_subnet_id: "subnet-0123456789abcdef0"  # Replace with your subnet ID
        assign_public_ip: yes  # This will assign a public IP to the instance

    - name: Fetch the public IP of the EC2 instance
      ec2_instance_facts:
        region: "us-east-1"
        instance_ids: "{{ ec2.instances[0].id }}"
      register: ec2_facts

    - name: Show the public IP of the EC2 instance
      debug:
        msg: "The public IP of the new EC2 instance is {{ ec2_facts.instances[0].public_ip }}"
~
```

Step 3: Update Jenkinsfile with EC2 Public IP

1. **Get EC2 Public IP**:
   - After the EC2 instance is created, get the public IP address.
2. **Update Jenkinsfile**:
   - Modify the `Jenkinsfile` to use the EC2 public IP for deployment.

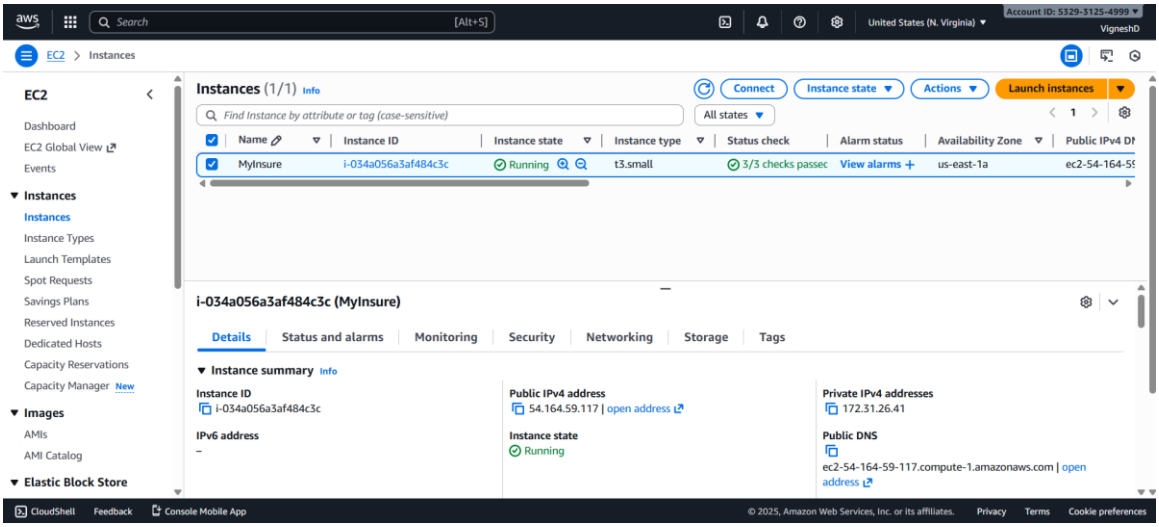
```jenkinsfile
pipeline {
    agent any

    environment {
        // Docker Hub credentials (stored in Jenkins)
        DOCKER_HUB_CREDENTIALS = credentials('dockerhub-cred')
        DOCKER_HUB_USER = "${DOCKER_HUB_CREDENTIALS_USR}"
        DOCKER_HUB_PASS = "${DOCKER_HUB_CREDENTIALS_PSW}"

        // Docker image names
        FRONTEND_IMAGE = "${DOCKER_HUB_USER}/insurance-frontend:latest"
        BACKEND_IMAGE = "${DOCKER_HUB_USER}/insurance-backend:latest"

        // AWS EC2 instance details
        EC2_HOST = "ubuntu@54.164.59.117"   // ✅ Replace with your EC2 public IP
```

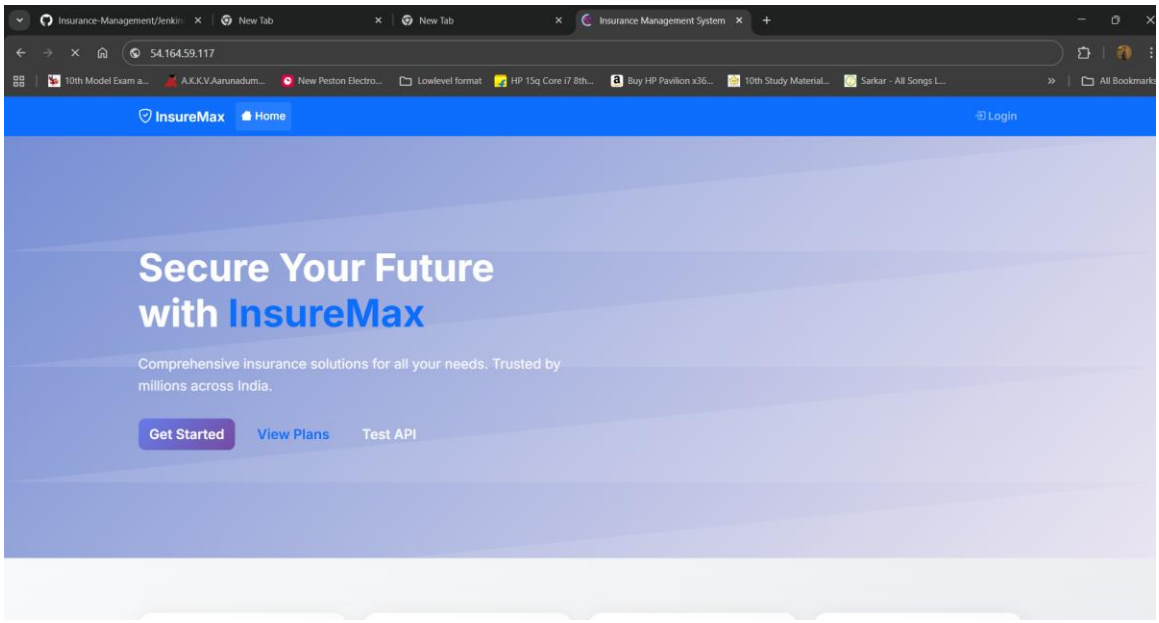


Build Pipeline In Jenkins

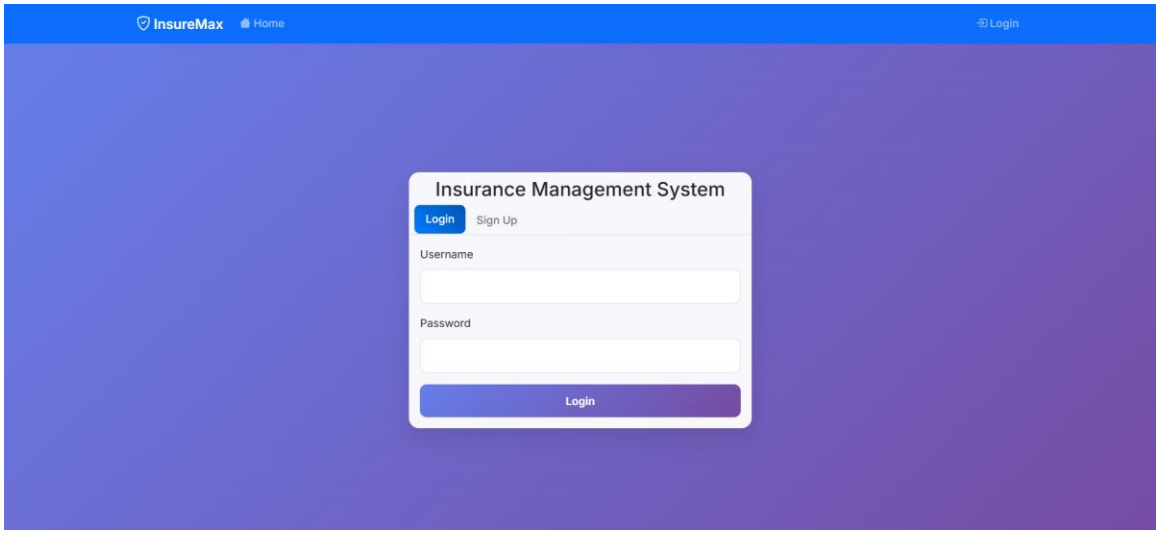
Final Step : Check AWS Instance Page

1. **Get EC2 Public IP**:
   - After the EC2 instance is created, get the public IP address.

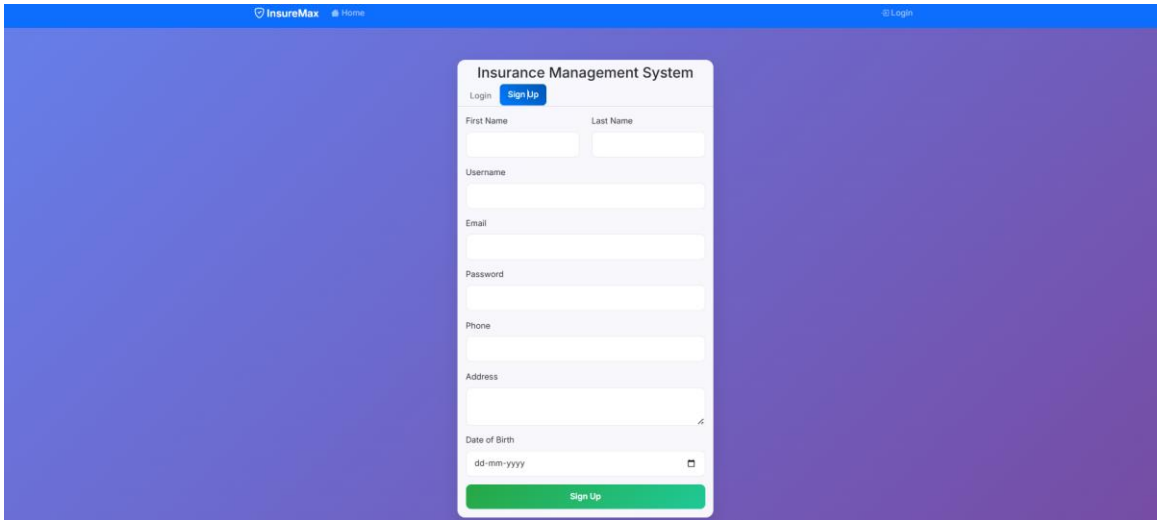


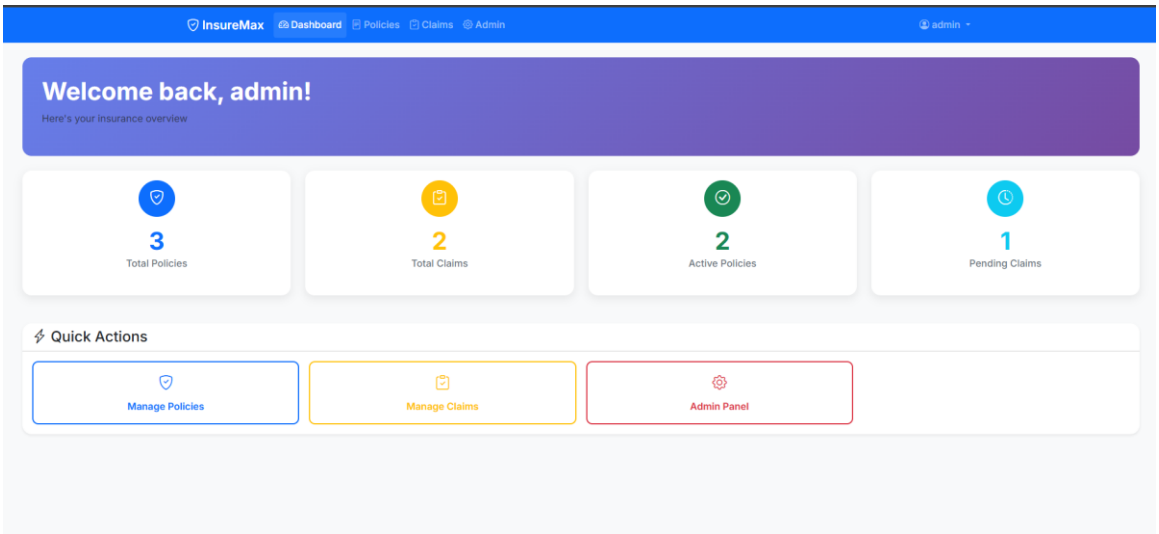


Application Deployed in AWS Running in IP : 54.164.59.117



Login Page





Overall Flow

1. **Backend**: **H2 Database** is used in-memory for testing and development. This simplifies database management and avoids the need for an external database during the development phase.
    - Dockerized Spring Boot application is built and pushed to a container registry (Docker Hub or AWS ECR).
    - Jenkins CI/CD pipeline automates the process of building, testing, and deploying the backend to AWS EC2 instances.
2. **Frontend**:
    - Angular application is built and served using Nginx or similar in a Docker container.
    - Docker Compose is used to manage both frontend and backend containers.
3. **Jenkins Pipeline**:
    - Jenkins handles the entire build, test, and deployment process through a pipeline defined in the `Jenkinsfile`.
    - The backend is deployed first, followed by the frontend application to ensure the system is up and running.
4. **Ansible for EC2 Creation**:
    - Ansible playbooks automate the provisioning of AWS EC2 instances.
    - The EC2 instance is set up, and the public IP is updated in the Jenkins pipeline for deployment.

Conclusion:

Each step should now be captured with screenshots that correspond to the terminal commands and actions performed. This guide covers everything from project setup to deploying your application on AWS EC2 through Jenkins CI/CD automation.